Manual técnico

Sistema de Control de Calidad Ancor · Instalación, arquitectura, reglas y mantenimiento

Versión 1.0 · 10 de octubre de 2026 · America/Santiago

1 Alcance y arquitectura

Este manual describe el código local al 10 de octubre de 2026. Está dirigido a quienes desarrollan y administran el proyecto. Es una guía de desarrollo y puesta en marcha local; todavía falta validar correo, usuarios y despliegue real.

Capa	Tecnología y responsabilidad
Frontend	React 18.3 y Vite 5. Interfaz demo, formularios y pantalla operativa conectada.
Backend	Django 6.0, Django REST Framework y Simple JWT. Validación, permisos, procesos SDI y archivos.
Persistencia	SQLite en config.local_settings; PostgreSQL en config.settings; SQLite en memoria en config.test_settings.
Documentos	pypdf para extracción de texto; archivos en MEDIA_ROOT con descargas autenticadas.
Correo	Proceso IMAP de recepción y bandeja de salida SMTP configurable; consola por defecto.

Dos modos con datos diferentes

App permite alternar demostración y sistema conectado. DemoApp conserva registros y preferencias en localStorage. OperationalApp inicia sesión con JWT y consulta la API. Los identificadores localId y usuarios ficticios de la demo no pueden reutilizarse como IDs del backend.

El panel completo, Equipo, Configuración, Ayuda y la exportación CSV pertenecen a la demo. El modo conectado ofrece su propia lista y detalle de SDI con indicadores y acciones operativas. No todas las pantallas del prototipo están conectadas a la API.

Recorrido de una petición

Navegador → proxy /api de Vite → autenticación y vista Django → serializer/permisos → servicios de dominio → base de datos, historial y cola de avisos. El envío de correo ocurre posteriormente mediante un proceso separado.

Contenido

2 mapa del código y relaciones; 3 instalación local; 4 reglas de datos; 5 ciclo y permisos; 6 API; 7 correo y PDF; 8 interfaz y CSV; 9 pruebas, mantenimiento y pendientes.

Fuentes: frontend/package.json, backend/requirements.txt, config/settings.py, local_settings.py, test_settings.py y ambos README.

2 Mapa del código y relaciones

Archivo o carpeta	Dónde intervenir o consultar
frontend/src/App.jsx	Modo, demo, navegación, filtros, sesión local y almacenamiento.
frontend/src/pages/DashboardPage.jsx	Resumen del panel; dashboardData.js define fecha y conteos.
frontend/src/components/QualityForms.jsx	SDIModal, ProductoModal, InspeccionModal y campos compartidos.
frontend/src/pages/OperationalApp.jsx	Login, recepción, productos, asignación, resultados y cierre conectados.
frontend/src/pages/workflowAvailability.js	Disponibilidad de acciones según rol, asignación y resultados.
frontend/src/api/workflow.js	JWT, solicitudes, renovación de tokens y descargas protegidas.
frontend/src/pages/csvExport.js	Columnas, relaciones y serialización del CSV demo.
backend/quality/models.py y serializers.py	Tablas, campos, relaciones y validación de entrada/salida.
backend/quality/workflow.py y services.py	Recepción, asignación, resultados, revisión, cierre y conservación.
backend/quality/workflow_api.py, views.py y urls.py	Endpoints, carga/descarga y coordinación de servicios.
backend/quality/permissions.py	Perfiles activos, roles y acceso a inspecciones.
backend/quality/mail.py y pdf_intake.py	Correo y lectura conservadora del PDF.

Relaciones principales

Un User tiene un PerfilUsuario con rol, área y estado activo. Una SDI tiene varios ItemSDI; cada producto tiene varias InspeccionSDI. Cada inspección posee una revisión visual y puede tener evidencias y concesiones. La SDI reúne documentos, eventos HistorialSDI y NotificacionSDI.

El catálogo Area describe el área del inspector, no el departamento de Producción/Bodega. area_inspector es un dato de salida de solo lectura: procede del inspector de la SDI para el producto y del inspector propio para la inspección.

ThemeToggle.jsx, styles.css y workflow.css controlan el tema. useAnimatedValues.js controla las transiciones de cifras. Las otras páginas de la demo permanecen en App; no hay React Router.

3 Instalación y arranque local

Requisitos: Python 3.12 o posterior y Node.js con npm. El proyecto dispone de una .venv local; no es necesario recrearla si ya está instalada. Ejecuta estos comandos en PowerShell desde la raíz del repositorio.

Backend con SQLite

```
cd backend
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe manage.py migrate --settings=config.local_settings
```

local_settings utiliza backend/local.sqlite3 y evita conectarse a PostgreSQL. Aplica todas las migraciones: 0004 incorpora el flujo/documentos/avisos; 0005 conserva asignaciones anteriores; 0006 incorpora anulación y archivo.

Cuentas operativas

Actualmente la base local no tiene usuarios. Los siguientes comandos son ejemplos para crear cuentas nuevas; piden la contraseña y su repetición en consola. No hay credenciales universales de la demo para el modo conectado.

```
.\.venv\Scripts\python.exe manage.py setup_quality_user supervisor --role supervisor `
    --settings=config.local_settings
.\.venv\Scripts\python.exe manage.py setup_quality_user inspector --role inspector --area `
    Calidad --settings=config.local_settings
```

Cada comando crea User y PerfilUsuario; --area crea o reutiliza un área. Si el nombre existe, el comando falla para no sobrescribirlo. El rol administrador del sistema de calidad no otorga por sí solo acceso al administrador Django.

Iniciar ambos servidores

```
.\.venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000 --settings=config.local_settings
```

En otra terminal desde la raíz:

```
cd frontend
npm.cmd install
npm.cmd run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

Abre http://127.0.0.1:5173. Vite reenvía /api a http://127.0.0.1:8000. API_PROXY_TARGET en frontend/.env.local permite cambiar el destino.

Las líneas largas de comandos deben copiarse completas desde el Markdown. Este procedimiento describe la instalación; las comprobaciones de esta edición se ejecutaron en el entorno existente.

4 Reglas de campos

La recepción puede conservar datos incompletos. Antes de asignar se exige completar y validar la solicitud y sus productos. Los controles del frontend ayudan a introducirlos; la API vuelve a comprobarlos.

Entidad y campo	Límite o comportamiento	Razón
SDI · activacion_proyecto	1 a 6 dígitos; conserva ceros iniciales.	Código textual, no cantidad.
SDI · numero_sdi	Solo dígitos; no tiene un máximo de longitud declarado.	Identificación de solicitud.
SDI · prefijo_bod	Booleano; desactivado por defecto.	Construcción del código.
SDI · fecha_emision	Desde 1 enero del año anterior hasta hoy en Santiago.	Regla acordada; impide años antiguos/futuros.
SDI · proyecto	Hasta 200 caracteres.	Capacidad del modelo.
SDI · departamento y solicitante	Hasta 150 caracteres cada uno.	Origen de la solicitud.
SDI · correo_origen	Correo válido; obligatorio para asignar.	Destinatario de los avisos.
SDI · prioridad	Baja, media o alta; media inicial.	Clasificación del trabajo.
Producto · articulo	Hasta 250; letras, números, espacios, puntos y guiones.	Nombre legible con signos limitados.
Producto · orden	Mayor orden de su SDI + 1; no editable.	Secuencia automática.
Producto · cantidad_indicada	Entero positivo opcional.	Conservar la cantidad recibida.
Producto · cantidad_a_inspeccionar	Entero >= 1; no supera indicada si se conoce.	Alcance autorizado.
Producto · codigo_plano	Hasta 150 caracteres; opcional.	Referencia técnica.
Inspección · cantidades	Inspeccionada <= restantes; aprobada de 0 a inspeccionada.	Evita liberar unidades de más.
Anulación · motivo	1 a 1000 caracteres después de quitar espacios extremos.	Justificar y conservar el evento.

A fecha de esta edición, el rango de emisión es 2025-01-01 a 2026-10-10; cambia automáticamente. Cantidad a inspeccionar utiliza --, -, + y ++ con pasos de 5 y 1 y mínimo 1. Cantidad aprobada admite 0. El campo Cantidad indicada conserva su entrada numérica propia.

Fuentes: models.py, serializers.py, workflow.py, QualityForms.jsx, sdiFields.js y productFields.js.

5 Ciclo operativo y permisos

Etapa	Condición y efecto
Recibir	Guarda SDI pendiente sin inspector, PDF original y propuesta. Solo gestión recibe y corrige.
Asignar	Exige código, fecha, proyecto, departamento, solicitante, correo, productos y un inspector activo. Pasa a en_proceso y crea una inspección por producto.
Inspeccionar	Exige asignación y SDI abierta. Completa siete criterios visuales; adjunta evidencias y finaliza cantidad aprobada.
Resolver	0 aprobadas rechaza; una parte produce parcial; el total requiere visual conforme. Recalcula producto y SDI.
Reinspeccionar	Último resultado rechazado/parcial, unidades restantes y ninguna inspección en curso. Mantiene el inspector asignado.
Concesión	Gestión registra responsable de Ingeniería y justificación; resuelve aprobación/rechazo. No hay rol de Ingeniería separado en esta interfaz.
Revisar y cerrar	Gestión revisa cuando todas las inspecciones terminan. Cierre exige revisión vigente, productos resueltos y ninguna concesión pendiente.

Visibilidad y autorización

Administrador, Jefe de Calidad y Supervisor gestionan SDI. Inspector ve únicamente las SDI que le han sido asignadas y trabaja sus propias inspecciones. Usuario y perfil deben estar activos. Los roles de gestión también pueden trabajar inspecciones conforme a los permisos del backend.

Los siete criterios son rebarbas, rayaduras, golpes, deformidad, pintura, burbujas y planitud. Cada uno admite conforme, no_conforme o no_aplica. Una modificación de resultados invalida la revisión previa; hay que revisar nuevamente antes de cerrar.

Estados y conservación

SDI: pendiente, en_proceso, aprobada, rechazada, concesion, cerrada o anulada. Producto añade parcial entre sus resultados; una inspección utiliza en_proceso, aprobada, parcial, rechazada o concesion. Cerrada conserva resultado_cierre; no equivale necesariamente a aprobada.

Retirar producto exige SDI pendiente sin asignación ni inspecciones y deja copia descriptiva en historial. Anular conserva código y documentos y es irreversible. Archivar oculta la SDI completa de listas habituales sin cambiar el estado; la API no convierte el archivo en un bloqueo operativo. Restaurar una anulada no la reactiva.

Las mutaciones de dominio usan transacciones y bloqueos de filas; las condiciones se revalidan en el servidor. No se permite DELETE de SDI o inspecciones.

6 API y autenticación

Las rutas siguientes usan el prefijo /api. Salvo la obtención/renovación de tokens, requieren Authorization: Bearer <access> y perfil activo. Los payloads ordinarios son JSON; PDF y evidencias usan multipart/form-data.

Método	Ruta sin /api	Uso
POST	token/ · token/refresh/	Credenciales username/password; renovación con refresh.
GET	me/ · dashboard/ · areas/	Perfil, indicadores y catálogo activo.
GET / POST	sdi/	Listar o crear SDI pendiente.
POST	sdi/recepcion/	Recibir archivo PDF y correo_origen.
GET / PATCH	sdi/{id}/	Detalle compuesto o corrección.
GET / POST	sdi/{id}/items/	Productos de una SDI.
GET / PATCH / DELETE	items/{id}/	Consultar, editar o retirar bajo condiciones.
POST	sdi/{id}/asignar/	inspector e instrucciones.
POST	sdi/{id}/revisar/ · cerrar/	Comentario y revisión/cierre.
POST	sdi/{id}/anular/ · archivo/	motivo; o archivar: true/false.
GET	sdi/{id}/historial/	Eventos por usuario y fecha.
GET	documentos/{id}/archivo/	PDF protegido.
GET / POST	items/{id}/inspecciones/	Inspecciones del producto.
POST	items/{id}/reinspecciones/	inspector, cantidad_inspeccionada y observaciones.
GET / PATCH	inspecciones/{id}/	Detalle y cambios permitidos.
POST	inspecciones/{id}/visual/ · aprobar/ · rechazar/	Criterios o cantidad_aprobada; resultados.
POST / GET	inspecciones/{id}/evidencias/ · evidencias/{id}/archivo/	Cargar / descargar evidencia.
GET / POST	inspecciones/{id}/concesiones/	Consulta y solicitud de concesión.
PATCH	concesiones/{id}/resolver/	decision: aprobada o rechazada.
GET / POST / PATCH	inspectores/ · inspectores/{id}/estado/	Catálogo, alta o activación según ruta.

GET sdi/ admite ?vista=activas, anuladas, archivadas o todas. La consulta conserva los permisos de cada usuario. Los IDs son enteros del servidor; codigo_sdi es un dato de salida, no una clave para sustituir el ID.

workflow.js guarda tokens en sessionStorage; nunca guarda la contraseña. Ante 401 intenta una renovación compartida y repite una sola vez. Si la renovación falla, borra la sesión. Los errores de validación y permiso llegan a la pantalla.

La tabla agrupa rutas; consulta quality/urls.py, workflow_api.py y views.py para el contrato completo. Los sufijos revisar/, cerrar/, etc. conservan el prefijo sdi/{id}/ o inspecciones/{id}/ indicado.

7 Correo y lectura del PDF

El proveedor aún no está definido. backend/.env.example documenta las variables, pero Django no carga ese archivo automáticamente: hay que exportarlas al proceso. Actualmente no se envía correo externo ni se simula una entrega exitosa.

Grupo	Variables y configuración
Recepción IMAP	SDI_IMAP_HOST, SDI_IMAP_USER, SDI_IMAP_PASSWORD o SDI_IMAP_OAUTH_TOKEN; SDI_IMAP_FOLDER (INBOX).
Remitentes	SDI_ALLOWED_SENDERS: direcciones separadas por coma; la restricción no reemplaza la validación del proveedor.
Envío SMTP	EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend; EMAIL_HOST, EMAIL_PORT, EMAIL_HOST_USER y EMAIL_HOST_PASSWORD.
Transporte	EMAIL_USE_TLS / EMAIL_USE_SSL según el servicio, sin activarlas de forma incompatible; DEFAULT_FROM_EMAIL revisado.
Estado actual	Backend de consola por defecto: avisos pendientes; hace falta SMTP y ejecutar el proceso.

Procesar entrada y salida

```
.\.venv\Scripts\python.exe manage.py process_sdi_mail --settings=config.local_settings
.\.venv\Scripts\python.exe manage.py process_sdi_mail --once --settings=config.local_settings
.\.venv\Scripts\python.exe manage.py process_sdi_mail --once --outbox-only `
    --settings=config.local_settings
```

El bucle consulta periódicamente los mensajes no leídos. Los adjuntos se registran antes de marcar el correo leído; no se borra del buzón. La deduplicación combina remitente, Message-ID y hash del PDF. Varios PDF de un mensaje pueden originar varias SDI.

Asignación y cierre crean avisos en la misma transacción del cambio. El envío posterior conserva errores e intentos, usa reintentos con espera creciente y Message-ID estable. Una interrupción entre aceptación SMTP y guardado puede duplicar un aviso; aceptación no prueba lectura del destinatario.

Límites de lectura y archivos

PDF: hasta 20 MB y 100 páginas, sin contraseña; mensaje de correo: hasta 25 MB. Se limita el contenido expandido por página a 10 MB y el texto extraído a 200 000 caracteres. Evidencias: hasta 20 MB.

pypdf extrae texto seleccionable y reconoce etiquetas conservadoras como Proyecto, Fecha, Producto/Artículo, Descripción y Cantidad. Se conserva la propuesta y el original. No hay OCR ni extracción externa mediante IA; las imágenes escaneadas requieren corrección manual. El ajuste a formatos de la empresa necesita PDF reales autorizados.

Fuentes: quality/mail.py, pdf_intake.py y management/commands/process_sdi_mail.py. En despliegue el proceso de correo necesita supervisión propia, además del servidor web.

8 Interfaz, almacenamiento y exportación

Panel y filtros

DashboardPage recibe estado y tabla mediante props de App. La cabecera calcula el día en America/Santiago y se actualiza cada minuto y al recuperar la pestaña. En proceso cuenta por separado estado de SDI, resultado_actual de producto y resultado de inspección. Excluye registros genéricos y no depende del periodo elegido.

El rendimiento por periodo y varios indicadores conservan datos de ejemplo. La búsqueda y los filtros reducen registros locales; Area usa el ID de area_inspector del catálogo, nunca el departamento solicitante. La demo sin JWT no inventa áreas; el modo conectado carga su catálogo real.

Tema, controles y movimiento

ancor.theme conserva la preferencia. ThemeToggle aplica data-theme y colorScheme; styles.css y workflow.css cubren ambos modos. La paleta oscura usa fondo #090909, superficies #171717 y #222222, texto #ededed y secundario #aaaaaa. Los colores de resultado y gráficos mantienen su función.

Icon.jsx aporta representaciones y descripciones; los controles incluyen etiquetas accesibles. QualityForms comparte campos de dígitos, fecha y cantidad. Cantidad usa pasos 1/5 y ajusta los saltos a sus límites.

useAnimatedValues anima cifras aproximadamente 420 ms desde el valor visible, incluso si cambia otra vez el periodo. Se cancela requestAnimationFrame al desmontar y se respeta movimiento reducido. Barras y etiquetas tienen sus propias transiciones en CSS.

Contrato CSV de la demo

csvExport.js exporta el listado filtrado y resuelve relaciones usando todos los registros. Emite 19 columnas: registro, tipo, artículo/actividad, SDI, producto, proyecto, departamento, ubicación antigua, área del inspector, responsable, prioridad, estado, vista, fecha/referencia, cuatro cantidades y observaciones.

Formato: UTF-8 con BOM, línea sep=;, separador punto y coma, finales CRLF, celdas entre comillas y comillas internas duplicadas. Los saltos de línea se conservan dentro de su celda. Se mantienen ceros, se dejan vacíos los datos ausentes y se normalizan fechas a AAAA-MM-DD. Una referencia ficticia como Hoy, 10:30 se conserva sin inventarle fecha.

Se antepone apóstrofo a cadenas que podrían interpretarse como fórmulas. No se exportan colores, bordes o anchos. La pantalla conectada todavía no tiene esta exportación.

El README del frontend, api/workflow.js, useAreaCatalog.js y pages/csvExport.js describen estas separaciones. Cambiar de modo no sincroniza ni traslada los datos locales.

9 Verificación, mantenimiento y pendientes

Comandos de comprobación

Desde frontend:

```
npm.cmd test
npm.cmd run build
```

Desde backend:

```
.\.venv\Scripts\python.exe manage.py test quality --settings=config.test_settings
.\.venv\Scripts\python.exe manage.py check --settings=config.local_settings
.\.venv\Scripts\python.exe manage.py makemigrations --check --dry-run `
    --settings=config.test_settings
```

Esta edición comprobó 37 pruebas del frontend, 28 del backend, compilación correcta y ausencia de migraciones nuevas por generar. Las pruebas del backend usan SQLite y archivos en memoria; no acceden al buzón ni a PostgreSQL. El hasher rápido de test_settings es exclusivo de pruebas.

Configuración de un entorno real

config.settings apunta a PostgreSQL control_calidad, usuario postgres, localhost:5433. Lee DB_PASSWORD, DJANGO_SECRET_KEY, DJANGO_DEBUG y DJANGO_ALLOWED_HOSTS del entorno. Antes de desplegar revisa esos valores, desactiva DEBUG y aplica las migraciones contra la base elegida.

El despliegue requiere servidor de aplicación, proxy del frontend hacia /api, HTTPS y proceso supervisado de correo. No uses runserver como servidor de producción ni sirvas MEDIA_ROOT como directorio público: las descargas pasan por endpoints autenticados.

Conservación y diagnóstico

- Respalda la base de datos y backend/media como conjunto. Verifica una restauración en un entorno separado; no se ha probado aquí una restauración completa.
- Ante errores de acceso, comprueba usuario y perfil activos, rol, asignación y propietario de la inspección antes de tocar datos.
- Ante avisos pendientes, revisa la configuración SMTP, ejecución de process_sdi_mail y errores/intentos visibles en la SDI.
- Para cambiar el contrato, revisa modelo, serializer, servicio, API, interfaz y pruebas relacionadas. Los controles visibles no sustituyen validación del servidor.

Validación pendiente

Crear usuarios reales, probar cada rol en navegador, configurar correo autorizado, usar muestras de PDF de la empresa y definir el circuito de Ingeniería. También falta conectar todas las pantallas y métricas demo a datos reales.

Referencias internas: ambos README, .env.example, requirements.txt, config/urls.py, quality/tests.py, test_workflow.py y frontend/tests. Las dependencias y comandos se documentan según este repositorio, no según una versión futura.